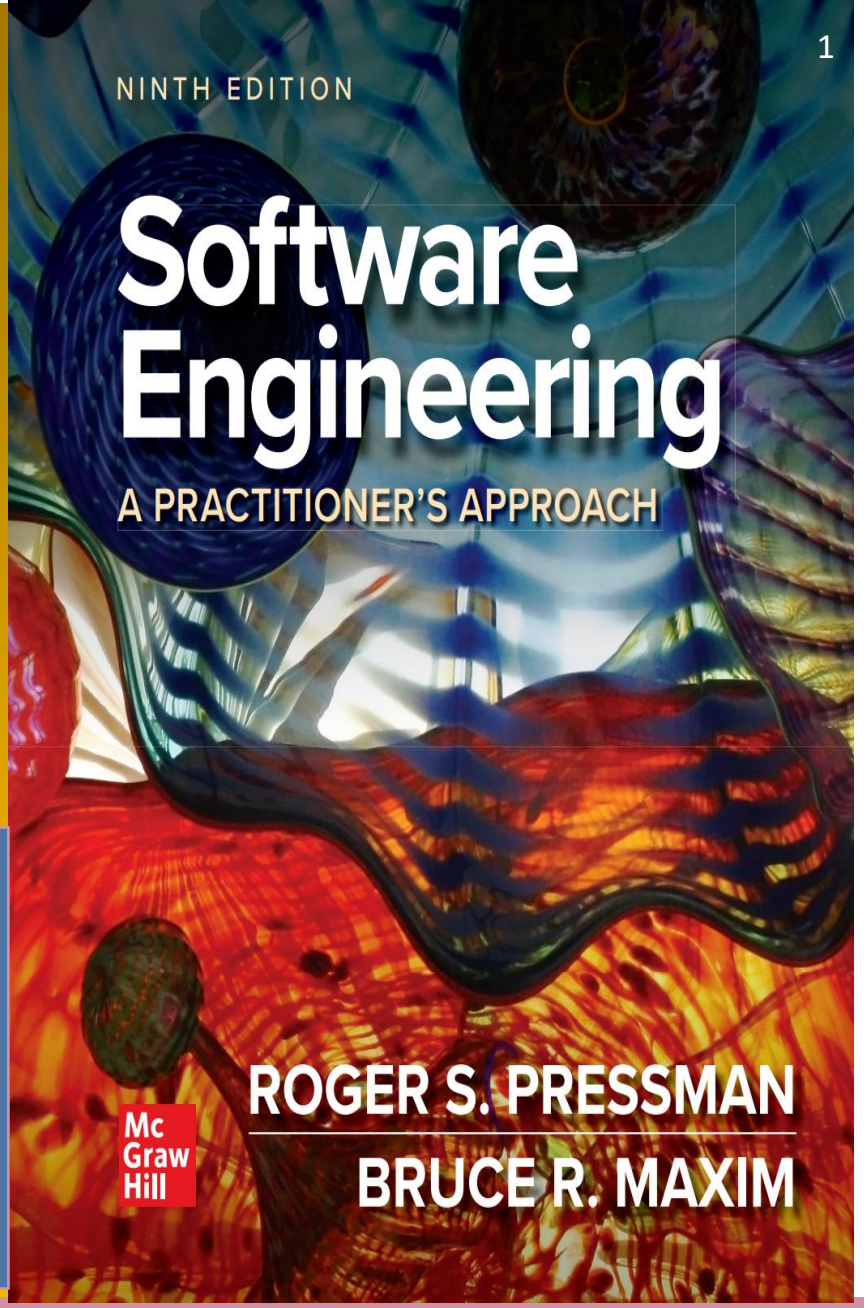
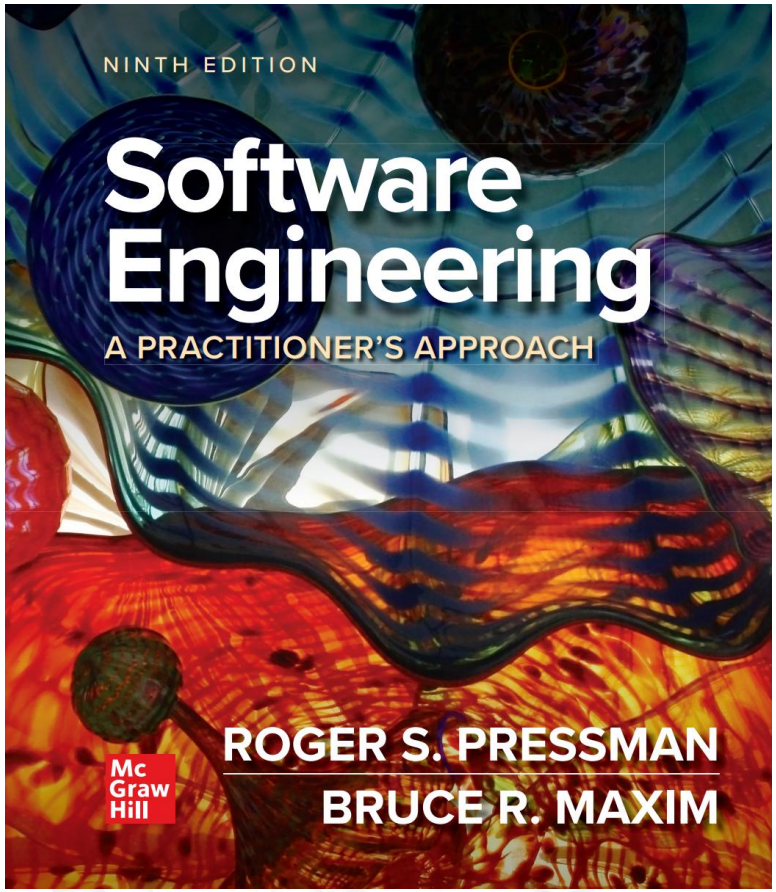
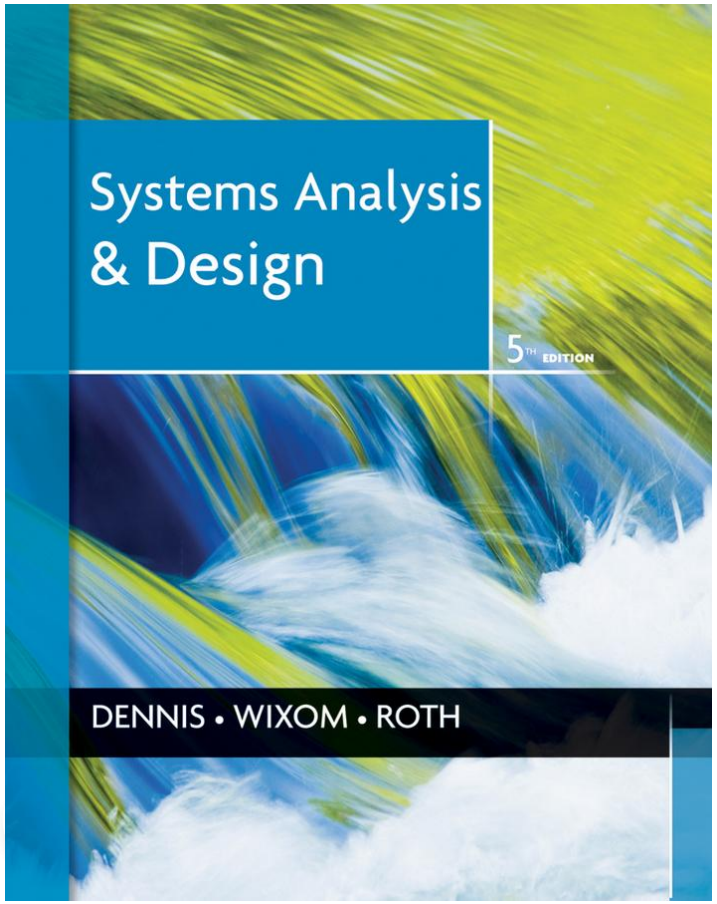
مهندسی نرم افزار



Website: ataghinezhad.github.io, Email:a0taghinezhad@gmail.com

منابع


NINTH EDITION
Software Engineering
A PRACTITIONER'S APPROACH
ROGER S. PRESSMAN
BRUCE R. MAXIM
Mc Graw Hill

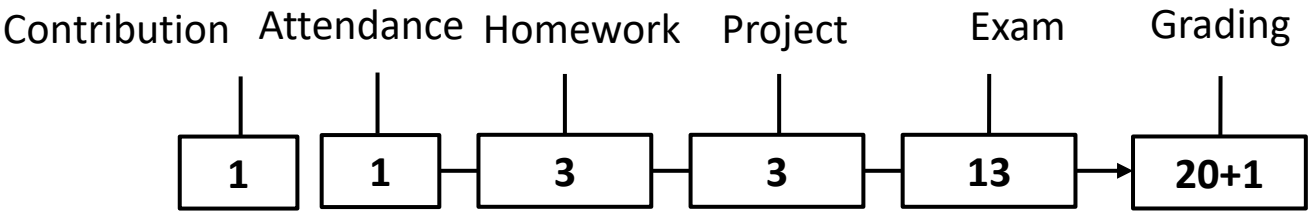

Systems Analysis & Design
5TH EDITION
DENNIS • WIXOM • ROTH

فهرست مطالب

- مقدمه‌ای بر مهندسی نرم‌افزار
- فرایندهای نرم‌افزار
- مدل سازی
  - مدل کردن نیازمندی‌ها
  - مفاهیم طراحی
- کیفیت و امنیت
  - تضمین کیفیت نرم افزار
  - نیازمندی‌های امنیتی نرم‌افزار
- مدیریت پروژه
  - مدل توسعه اسکرام و چابک
  - نمودار پرت و گانت
  - مفاهیم مدیریت پروژه

Dr. A. Taghinezhad

ارزیابی



Dr. A. Taghinezhad

ارزیابی پروژه

- ارائه پروژه صنعتی:
- نوع ارزیابی
- ۱:) کیفیت مطلب
- ۲) مفاد ارائه (اسلایدها، استفاده از تصاویر)
- ۳) کیفیت ارائه(نحوه بیان و ارائه جذاب)

Dr. A. Taghinezhad

مقدمه‌ای بر مهندسی نرم‌افزار

Dr. A. Taghinezhad

نرم‌افزار چیست ؟

- دستورات برنامه‌های کامپیوتری که در صورت اجرا شدن باعث انجام عمل و کارایی خواسته شده می‌شوند.
- ساختمان داده‌هایی که باعث می‌شود برنامه‌ها بطور مناسبی اطلاعات را دستکاری کنند.
- مستنداتی که توصیف کننده عملکرد برنامه‌ها هستند.
- در مفهوم عام در هر محیطی (کارخانه، آموزش و ...) مستقل از IT یک تفکر گرداننده سیستم موجود می‌باشد که سخت افزار موجود را وادار به حرکت می نماید.

Dr. A. Taghinezhad

نگاه اجمالی

مراحل کار چیست؟

مشتریان و سایر ذی‌نفعان، نیازهای خود را برای یک نرم‌افزار یا محصول مشخص می‌کنند. مهندسان نرم‌افزار بر اساس این نیازها، محصول نرم‌افزاری را توسعه داده و در نهایت کاربران مشخصی برای رفع یک نیاز خاص از آن استفاده می‌کنند یا به آن دسترسی پیدا می‌کنند.

محصول کار چیست؟

یک برنامهٔ کامپیوتری که در یکی از محیط‌های مشخص کار می‌کند و نیازهای یک یا چند کاربر نهایی را برطرف می‌سازد و خدماتی را ارائه می‌دهد.

چگونه مطمئن شویم که درست انجامش داده‌ام؟

اگر نرم‌افزار مهندسی است، ایده‌های مطرح‌شده را در باقی‌مانده این کتاب به‌کار می‌گیرد. اگر کاربر نهایی هستید، مطمئن شوید نیازها و محیط خود را به‌درستی درک کرده‌اید و **سپس** کاربردی مناسب را انتخاب کنید که هر دو را بهترین شکل برآورده کند.

Quick Look

کامپیوتر چیست؟

نرم‌افزار مجموعه‌ای از متخصصان است که محصولی نرم‌افزاری را طراحی می‌کنند و سپس در طولانی‌مدت از آن پشتیبانی می‌کنند. این نرم‌افزار در قالب برنامه‌هایی اجرا می‌شود که روی یک کامپیوتر و معماری‌های مختلف قرار می‌گیرند و اطلاعات را به‌صورت مجازی (دیجیتال) در اختیار کاربران قرار می‌دهند.

چه کسی آن را می‌سازد؟

مهندسان نرم‌افزار، به همراه سایر افراد فعال در حوزهٔ صنعت و فناوری، نرم‌افزار را توسعه می‌دهند و به‌صورت مستقیم یا غیرمستقیم در دنیای صنعتی از آن استفاده می‌کنند.

چرا مهم است؟

نرم‌افزار به دلیل تأثیر گسترده‌ای که در جنبه‌های مختلف زندگی ما دارد، تقریباً در همهٔ جنبه‌های تجارت، فرهنگ و فعالیت‌های روزمرهٔ ما نقش پررنگی ایفا می‌کند.

خصوصیات نرم افزار

۱- نرم افزار توسعه می یابد یا طراحی می شود، اما به مفهوم کلاسیک ساخته نمی شود.

گرچه شباهت هایی بین توسعه نرم افزار و ساخت سخت افزار وجود دارد، اما این دو فعالیت اساساً با هم متفاوتند.

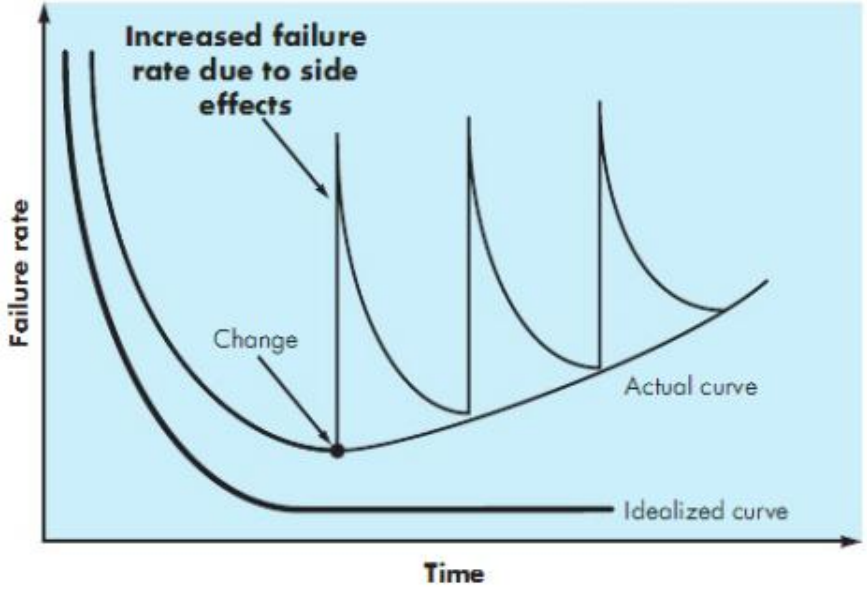
- ✓ در هر دو آنها کیفیت بالا حاصل طراحی خوب خواهد بود، اما **مرحله ساخت در مورد سخت افزار می تواند یک سری مشکلات کیفیتی داشته باشد** که در مورد نرم افزار وجود ندارد .
- ✓ هر دو فعالیت وابسته به مردم است اما **ارتباط بین افراد متخصص و کار** صورت گرفته کاملاً متفاوت است.
- ✓ هر دو مستلزم ساختن یک محصول هستند اما روش ها متفاوتند

Dr. A. Taghinezhad

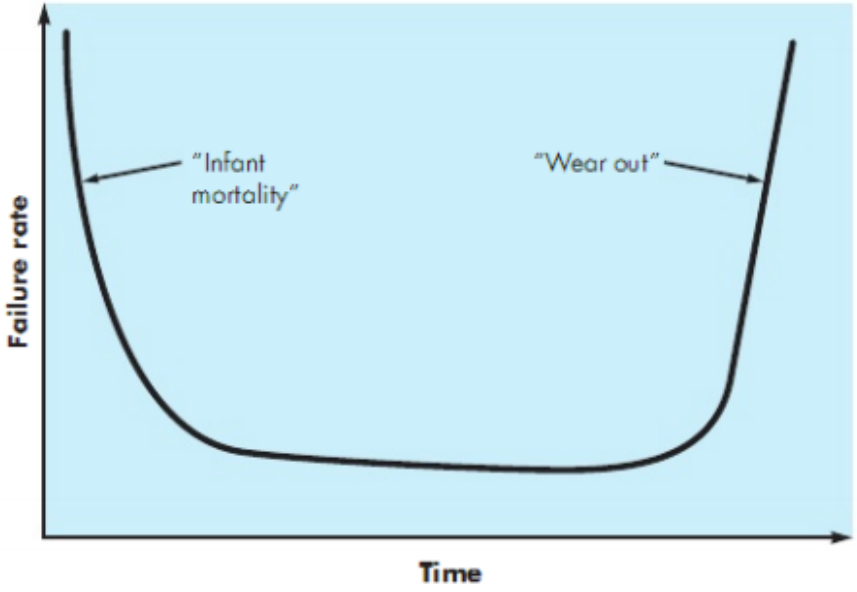
خصوصیات نرم افزار

۲- نرم افزار فرسوده نمی شود اما کیفیت خود را از دست می دهد.

سخت افزار در اوایل عمرش میزان عدم موفقیت نسبتاً بالایی دارد. نقایص اصلاح شده و میزان شکست برای مدتی به سطح ثابتی میرسد و با گذشت زمان، سخت افزار فرسوده می شود و نرخ شکست دوباره افزایش می یابد.



منحنی شکست نرم‌افزار



منحنی شکست سخت‌افزار

مهندسی نرم افزار:

مجموعه ای از تکنیک، قواعد، استانداردها و ابزار مهندسی به منظور تولید یک نرم افزار

۱- مقرون به صرفه (Benefit > cost)

۲- با کیفیت: ارضاء سطح نیازمندیهاست. یک مفهوم نسبیت است و از یک نرم افزار به نرم افزار دیگر متفاوت است.

Dr. A. Taghinezhad

مهندسی نرم افزار (ادامه):

$$Cost_{life\ cycle} = Cost_{Develop} + Cost_{maintenance}$$

- $Cost_{maintenance} = 4Cost_{Develop}$
- به‌طور میانگین ۷۰ درصد هزینه تولید نرم افزار مربوط به نگهداری آن است.
  - کیفیت بالای تولید و توسعه نرم افزار
  - بهتر شدن ابزارها و تجربه‌های ساخت نرم‌افزار
  - مطالعات جدیدتر درصد فعالیت‌های مربوط به تعمیر و نگهداری نرم‌افزار که مرتبط با رفع اشکال هستند را نزدیک به ٪۲۱ گزارش کرده‌اند.

Dr. A. Taghinezhad

مهندسی نرم افزار 2026(ادامه):

$$Cost_{life\ cycle}=Cost_{Develop}+Cost_{maintenance}$$

- هزینه نگهداری نرم‌افزار بخش قابل‌توجهی از هزینه چرخه عمر آن را تشکیل می‌دهد .
- مطالعات مختلف معمولاً سهم نگهداری و تکامل نرم‌افزار را حدود ۶۰تا ۸۰درصد از هزینه‌های چرخه عمر گزارش کرده‌اند؛ بااین‌حال، این مقدار به نوع سیستم، طول عمر، حوزه کاربرد و شیوه توسعه و بهره‌برداری وابسته است.
- نگهداری نرم‌افزار شامل فعالیت‌های اصلاحی (Corrective)، تطبیقی (Adaptive)، تکاملی/بهبوددهنده (Perfective)و پیشگیرانه (Preventive)است .در برآوردهای کلاسیک، نگهداری اصلاحی حدود ۲۰درصد از هزینه‌های نگهداری را تشکیل می‌دهد؛ برخی مطالعات نیز سهمی در حدود ۱۷تا ۲۱درصد را گزارش کرده‌اند .این عدد به معنی آن نیست که ۲۱درصد کل هزینه چرخه عمر صرف رفع اشکال می‌شود.

چرا اصول طراحی نرم‌افزار؟

نظریه مهندسی نرم‌افزار در سال ۱۹۶۸ در کنفرانس «بحران نرم‌افزار» مطرح گردید. در آن زمان تولید نرم‌افزارها به صورت غیر رسمی (سلیقه‌ای) بود. بنابراین نرم‌افزارهای تولید شده دارای اشکالات زیر بودند:

- انجام پروژه‌های بزرگ **سال‌های طولانی** طول می‌کشید.
- **هزینه واقعی** نرم‌افزار **بیش از مقادیر تخمین** زده بود. (هزینه تولید نرم افزار رو به افزایش بود)
- **عدم تطابق** نرم افزار تحویل شده با مشخصات تعیین شده.
- قابل اعتماد نبودند (**کیفیت پایین** نرم افزار).
- نگهداری آن‌ها دشوار بود (**نگهداری پر هزینه** نرم افزار).
- نداشتن سنجش میزان پیشرفت کار (۹۰ خط از ۱۰۰ خط برنامه ۹۰٪ برنامه نیست).
- پیشرفت سریع سخت افزار

مهندس نرم افزار

- مهندس نرم افزار کسی است که بتواند از ابزارها و تکنیک های موجود به کمک علم خود بهره گیرد و پس از تجزیه و تحلیل مساله آن را پیاده سازی و مدیریت نماید.

Dr. A. Taghinezhad

دوره تکامل نرم افزار

- دوره اول ۱۹۶۵ – ۱۹۵۰
  - نرم افزار های تک کاربره بودند
  - تمام فرآیند توسعه نرم افزار متکی به یک نفر بود.
  - یک نفر هم نرم افزار را توسعه میداد هم اجرا و هم پشتیبانی میکرد.

Dr. A. Taghinezhad

دوره تکامل نرم افزار (ادامه)

- دوره دوم ۱۹۷۵ – ۱۹۶۵
  - ایده چند برنامه ای (multi programming) و سیستم های چند کاربره (multi User)
  - روش های محاوره‌ای در نرم افزار unix os توسعه سیستم های بلادرنگ و تولید خروجی در چند میلی ثانیه به جای چند دقیقه
  - نرم افزار ها کمی پیچیده تر شدند و انبوهی از نرم افزارهای بدون استاندارد تولید شدند; زبان برنامه نویسی Basic
    - اولین نسل از سیستمهای مدیریت بانک اطلاعات
  - بحران نرم افزار به وقوع پیوست.

دوره تکامل نرم افزار (ادامه)

- دوره سوم ۱۹۸۵ – ۱۹۷۵
  - ریز پردازنده‌های کامپیوترهای شخصی کاربرد گسترده‌ای پیدا کرد. Intel 8080
  - توسعه و مطرح شدن سیستم‌های تعبیه شده (Embeded).
  - توسعه‌ی شبکه‌های محلی (LAN)
  - نرم افزار ها به خود حالت توزیع شده گرفتند.

Dr. A. Taghinezhad

دوره تکامل نرم افزار (ادامه)

- دوره چهارم ۱۹۸۵ – ۲۰۱۰
  - مفهوم برنامه نویسی شی گرا
  - قابلیت استفاده مجدد از مولفه ها
  - پیدایش نرم افزارهای خبره و پردازشات موازی

Dr. A. Taghinezhad

دوره تکامل نرم افزار (ادامه)

- **دوره پنجم :از حدود ۲۰۱۰ تا ۲۰۲۰**
- با ظهور رایانش ابری، کلان‌داده و زیرساخت‌های نرم‌افزاری جدید، نرم‌افزار بیش از پیش به یک **خدمت مقیاس‌پذیر و مبتنی بر داده** تبدیل شد.
- **رایانش ابری :**اجرای نرم‌افزار و ذخیره‌سازی داده روی زیرساخت‌های ابری و امکان مقیاس‌پذیری پویا.
- **کلان‌داده** : پردازش و تحلیل حجم عظیمی از داده‌ها با زیرساخت‌های توزیع‌شده.
- **هوش مصنوعی و یادگیری ماشین :**استفاده از مدل‌های یادگیری برای پیش‌بینی، طبقه‌بندی، توصیه و تصمیم‌گیری.
- **میکروسرویس‌ها:** تجزیه سیستم‌های بزرگ به سرویس‌های مستقل و قابل توسعه.
- **DevOps و CI/CD :**یکپارچه‌سازی توسعه و عملیات و حرکت به سمت استقرار و به‌روزرسانی مداوم.
- **نرم‌افزار به‌عنوان سرویس (SaaS) :**ارائه نرم‌افزار به‌صورت یک خدمت آنلاین به جای نصب و مدیریت محلی.
- افزایش اهمیت **امنیت سایبری، حریم خصوصی و تاب‌آوری نرم‌افزار.**
- گسترش معماری‌های توزیع‌شده، کانتینرها و سیستم‌های مقیاس‌پذیر.

دوره تکامل نرم افزار (ادامه)

دوره ششم :از حدود ۲۰۲۰به بعد؛ عصر نرم‌افزارهای مولد و عامل‌محور

- **هوش مصنوعی مولد (Generative AI)** :تولید متن، کد، تصویر و سایر محتواها
- **مدل‌های زبانی بزرگ (LLMs)** :تبدیل هوش مصنوعی به بخشی از نرم‌افزار
- **عامل‌های هوش مصنوعی (AI Agents)** :تفسیر هدف، برنامه‌ریزی و اجرای خودکار وظایف
- **استفاده از ابزارها و APIها** :تعامل با پایگاه‌داده، وب، سیستم‌عامل و سرویس‌ها
- **حافظه و زمینه (Memory & Context)** :استفاده از اطلاعات تعاملات و وضعیت قبلی
- **اجرای چندمرحله‌ای** :شکستن هدف به مجموعه‌ای از اقدامات وابسته
- **سیستم‌های چندعاملی (Multi-Agent)** :همکاری چند عامل برای حل مسائل پیچیده
- **همکاری انسان و عامل** :نظارت و تأیید اقدامات حساس توسط انسان
- **توسعه نرم‌افزار با هوش مصنوعی** :تولید، تست، بازبینی و نگهداری کد با کمک هوش مصنوعی
- **رابط‌های هدف‌محور** :تعامل با نرم‌افزار از طریق زبان طبیعی و بیان هدف

Dr. A. Taghinezhad

دوره	سال‌ها	ویژگی‌های اصلی	مثال‌های واقعی
دوره اول	۱۹۵۰– ۱۹۶۵	•نرم‌افزارهای تک‌کاربره •توسعه وابسته به افراد محدود •برنامه‌های بسیار تخصصی	• IBM 701، IBM 704 • FORTRAN (1957) •سیستم‌های اولیه محاسبات علمی و نظامی
دوره دوم	۱۹۷۵-۱۹۶۵	-چندبرنامه‌ای - (Multiprogramming) سیستم‌های چندکاربره - (Multi-user) نرم‌افزارهای محاوره‌ای -سیستم‌های بلادرنگ -اولینDBMS ها -بحران نرم‌افزار	- OS/360 (IBM) - UNIX (1969) -زبان BASIC (1965) - CTSS (MIT) -سیستم کنترل ترافیک هوایی IDS (1964), IMS - - (1966)کنفرانس ناتو ۱۹۶۸ طرح اصطلاح Software Engineering
دوره سوم	۱۹۸۵-۱۹۷۵	-ظهور ریزپردازنده‌ها -گسترش کامپیوترهای شخصی - (PC) سیستم‌های تعبیه‌شده - (Embedded)شبکه‌های محلی- (LAN) نرم‌افزارهای توزیع‌شده	Intel 8080 (1974) -، - IBM PC (1981) - Apple II (1977) ماشین‌حساب‌هایTI ، سیستم ترمز ABS - Ethernet (1973, Xerox PARC، - IEEE 802.3 (1983) Novell NetWare - NFS (1984, Sun - (Microsystemsپایگاه داده Oracle (1979)

دوره	سال‌ها	ویژگی‌های اصلی	مثال‌های واقعی
دوره چهارم	۱۹۸۵– تا ۲۰۱۰)	-شی‌گرایی - (OOP) بازاستفاده از مولفه‌ها - (Components & Patterns)سیستم‌های خبره - (Expert Systems) پردازش موازی و توزیع‌شده	-زبان (1985) ++C، Java (1995) - Design Patterns (1994)، معماری SOAو Microservices - MYCIN، XCON - Cray Supercomputers - MPI (1994) - GPU parallel computing
دوره پنجم (پیشنهادی)	۲۰۱۰-۲۰۲۰	-رایانش ابری - (Cloud) هوش مصنوعی و یادگیری ماشین -کلان‌داده - (Big Data) SaaSو - DevOps امنیت سایبری و بلاک‌چین	- AWS, Azure, Google Cloud - TensorFlow (2015)، - ChatGPT (2022) Hadoop, Spark - GitHub, Docker, Kubernetes - Salesforce (SaaS) - Bitcoin (2009), Ethereum
دوره ششم	۲۰۲۰–تاکنون	•هوش مصنوعی مولد •مدل‌های زبانی بزرگ (LLMs) **AI Agents وMulti-Agent Systems** •نرم‌افزارهای هدف‌محور و ابزارمحور •تعامل با زبان طبیعی	• ChatGPT (2022) • GitHub Copilot • AI Coding Agents • Agentic Workflows •سیستم‌های چندعاملی

Dr. A. Taghinezhad

داستان نرم‌افزار

- بسیاری از عوامل ایجاد مشکلات نرم افزاری از طریق علم داستان سرایی که در اوایل تاریخ توسعه نرم افزار بوجود آمد، دنبال می شود.
- داستانهای نرم افزاری اطلاعات غلط و گیج کننده ای را انتشار دادند.
- امروزه اکثر افراد حرفه ای این داستانها را حاوی صفات گمراه کننده ای می دانند که مشکلات زیادی را برای مدیران و افراد متخصص ایجاد می نماید.

- انواع اشتباهات، تصورات و داستانهای نرم افزاری

1. داستانهای مدیریتی
2. داستانهای مشتری
3. داستانهای متخصصین و مجریان

Dr. A. Taghinezhad

داستان نرم‌افزار (ادامه)

- **داستانهای مدیریتی**
- مدیران مسئول نرم افزار، مانند اکثر مدیران دیگر، اغلب برای مواردی از قبیل تامین بودجه،جلوگیری از لغزش زمانبندی پروژه و ارتقاء کیفیت، تحت فشارند.
- داستان: قبلا کتابی در اختیار داشتیم که حاوی استانداردها و رویه هایی برای تولید نرم افزار بود ولی آیا کتاب تمام آنچه را افراد نیاز دارند، در اختیار دارد.
- واقعیت : (در بسیاری از موارد جواب سئوالات زیر خیر است)
- کتاب استانداردها می تواند وجود داشته باشد

1. آیا افراد از این کتاب مطلع هستند؟
2. آیا از کتاب استفاده می شود؟
3. آیا کتاب ، منعکس کننده مهندسی نرم افزار مدرن است؟
4. آیا کتاب کامل است؟
5. آیا کتاب ، زمان تولید نرم افزار را کاهش می دهد؟
6. آیا کتاب ، باعث ارتقاء سطح کیفی نرم افزار می شود؟

Dr. A. Taghinezhad

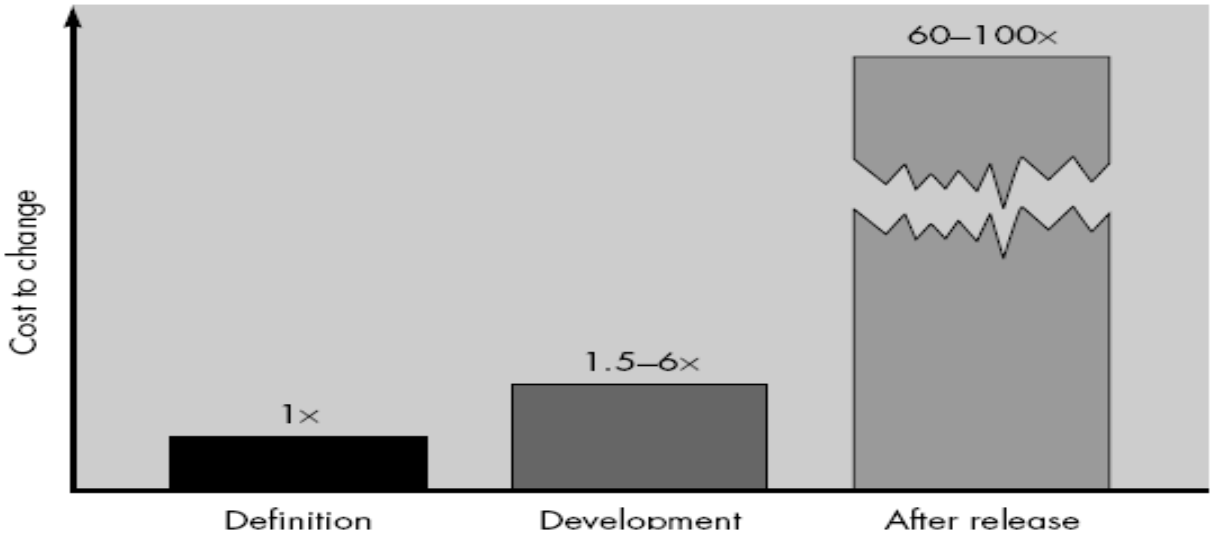
داستان نرم‌افزار (ادامه)

- **داستانهای مشتری**
- مشتریانی که تقاضای سیستم نرم افزاری دارند، فرد همکار، یک گروه کاری، واحد بازاریابی یا فروش شرکت خارجی باشند که نرم افزار را از طریق قرارداد درخواست نموده اند.
- داستان: نیازسنجی (مهندسی نیاز) پروژه دائم در حال تغییر است، اما این تغییرات به راحتی سازماندهی می شوند، زیرا نرم افزار قابل انعطاف است.
- واقعیت : با اطمینان میتوان گفت که نیازسنجی نرم افزار تغییر میکند، اما برخورد با این تغییر برحسب زمان بیان آن متفاوت خواهد بود.
-

Dr. A. Taghinezhad

داستان نرم‌افزار (ادامه)

- تاثیر تغییرات نیازهای کاربران نسبت به زمان، متفاوت است



- هر تغییر باعث افزایش ناگهانی درخواست برای منابع بیشتر و اصلاحات عمده، بنابراین افزایش هزینه را به دنبال دارد که این افزایش در مراحل (فازهای) مختلف تولید و توسعه نرم افزار متفاوت می باشد.

داستان نرم‌افزار (ادامه)

- **داستانهای متخصصین**
- پس از گذشت ۵۰ سال از بحران نرم افزار، هنوز یکسری متخصصین به داستان های زیر اعتقاد دارند.
- داستان: وقتی نرم افزار تولید و به مشتری تحویل داده شد،کارمان تمام شد.
- واقعیت: آمار نشان داده که ۶۰ تا ۸۰ درصد از کل تلاشهای صورت گرفته برای تولید نرم افزار، تازه بعد از آنکه به دست مشتری می رسد، انجام می شود.

- داستان:تا زمان اجرای نرم افزار، راهی برای بررسی کیفیت آن وجود ندارد.
- واقعیت: یکی از موثرترین مکانیزمهای تضمین کیفیت نرم افزار،که میتوان از شروع پروژه بکار گرفت، بازبینی فنی رسمی است که بعنوان فیلترهای کیفیتی محسوب میشود و موثرتر از تست نرم افزار برای یافتن گروه های خاصی از خطاها و نقایص نرم افزار می باشد.

انواع نرم‌افزار

سیستمی

- مستقیماً با سخت افزار در ارتباط اند؛ سیستم عامل ، کامپایلرها
- مجموعه‌ای از برنامه‌هایی که برای دادن سرویس به برنامه‌های دیگر نوشته شده‌اند.

علمی

- انجام امور علمی، فنی و مهندسی؛ Mathimatica, CAD
- این نرم‌افزارها الگوریتم‌های خاصی را روی اعداد اجراء می‌کنند.

کاربردی و تجاری

- عملاً متکی به یک پایگاه داده می باشند. (پردازش تراکنش بانکی)
- برنامه مستقل هستند که برای حل یک نیاز شغلی مورد استفاده قرار می‌گیرند.

هوش مصنوعی

- این گونه نرم‌افزارها از الگوریتم‌های غیر عددی برای حل مسائل استفاده می‌نمایند.
- سیستم‌های خبره، تشخیص الگو، شبکه عصبی مصنوعی و ...

وب

- صفحات وب که با استفاده از مرورگرها بازیابی می‌شوند.
- این نرم‌افزارها بر روی کامپیوترهایی قرار دارند که به شبکه‌ها متصل هستند.

و ....

- ...

نیازمندیها (Requirement)

- Functional (عملیاتی – کارکردی):
  - ماهیتی کسب و کار دارد. در واقع به کارکردهایی از Business که بایستی در یک نرم افزار Automate گردد.
  - تصویری از **فرآیندهای کسب و کار** سازمان
  - مثلاً در یک نرم افزار مالی توانایی محاسبه تراز مالی ، سند زنی و...
  - در یک نرم افزار آموزشی امکان ثبت نام، انتخاب واحد، حذف اضافه، صدور کارنامه ، ....

Dr. A. Taghinezhad

نیازمندیها (Requirement) ادامه

- Non- functional (غیر عملیاتی – غیر کارکردی):

  - این نیازمندیها به خصوصیات کیفی و استانداردها مانند امنیت (Security)، کارایی (Performance) دسترس پذیری، زمان پاسخ.
  - مستقیماً به کسب و کار مربوط نیست و تعدادش از یک نرم افزار به نرم افزار دیگر متفاوت است.

Dr. A. Taghinezhad

Software engineering is a layered technology





لایه تاکید بر کیفیت

- این لایه نشان می دهد که یک محصول نرم افزاری باید از یک سری استاندارد که متضمن کیفیت (Quality) بالای محصول است تبعیت میکند. بدیهی است که هیچ شرکت نرم افزاری دوست ندارد نرم افزار بی کیفیت تحویل مشتری دهد.

Dr. A. Taghinezhad

لایه تاکید بر کیفیت

- کیفیت نرم‌افزار به دو رده مرتبط اما مجزای زیر اشاره دارد:

۱) کیفیت **عملیاتی نرم‌افزار** (Software Functional Quality):
شاخصی جهت نشان دادن میزان تطابق نرم‌افزار با نیازمندی‌های عملیاتی تعریف شده برای نرم‌افزار است.

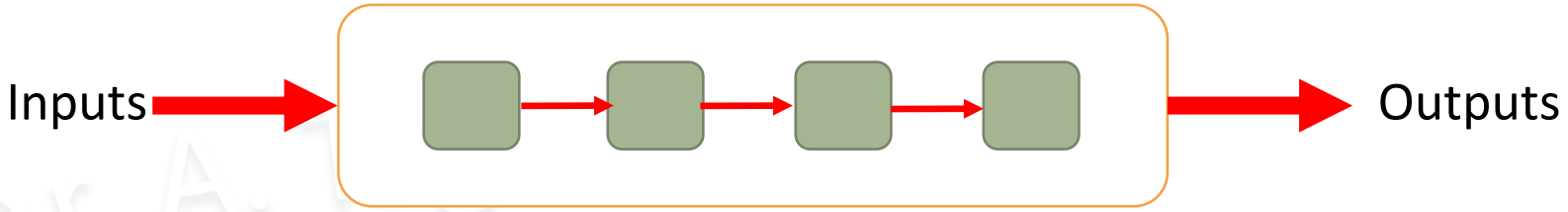
۲) کیفیت **ساختاری نرم‌افزار** (Software Structural Quality):
که منعکس کننده میزان دست یابی به نیازمندی‌های غیر عملیاتی مانند استحکام (Robustness) و قابلیت نگهداری (Maintainability) نرم‌افزار است.

Dr. A. Taghinezhad

لایه فرآیندها

- اساس مهندسی نرم افزارفرآیند می باشد. **لایه فرآیند مانند چسبی است که لایه های فناوری راباهم** نگه داشته.
- این لایه خود در برگیرنده لایه تکنولوژی‌های که در طی زمان توسعه نرم‌افزار را فراهم می‌کند.

- تعریف فرآیند: دنباله‌ای (نه مجموعه) از فعالیت ها که برای یک هدف خاص تعریف می شود



فرآیند توسعه نرم افزار Software Development Process

- لایه فرآیند یک چارچوبی را برای تحویل موثر نرم افزار مهندسی تعریف می‌کند.
- این لایه **اساس کنترل مدیریت پروژه های نرم افزاری** را شکل می دهد و مفادی را که در آن **روشهای فنی** (Methods) اعمال میشوند را ایجاد کرده و **محصولات کاری** (**مدل** ها، اسناد، داده ها، گزارش ها، فرم ها و غیره) تولید می شوند،
  - **نقاط قوت ایجاد می شوند، کیفیت تضمین شده** است، و تغییرات را به درستی مدیریت میکند.

Dr. A. Taghinezhad

شکل ۲.۲ جریان‌های فرایند توسعه نرم‌افزار

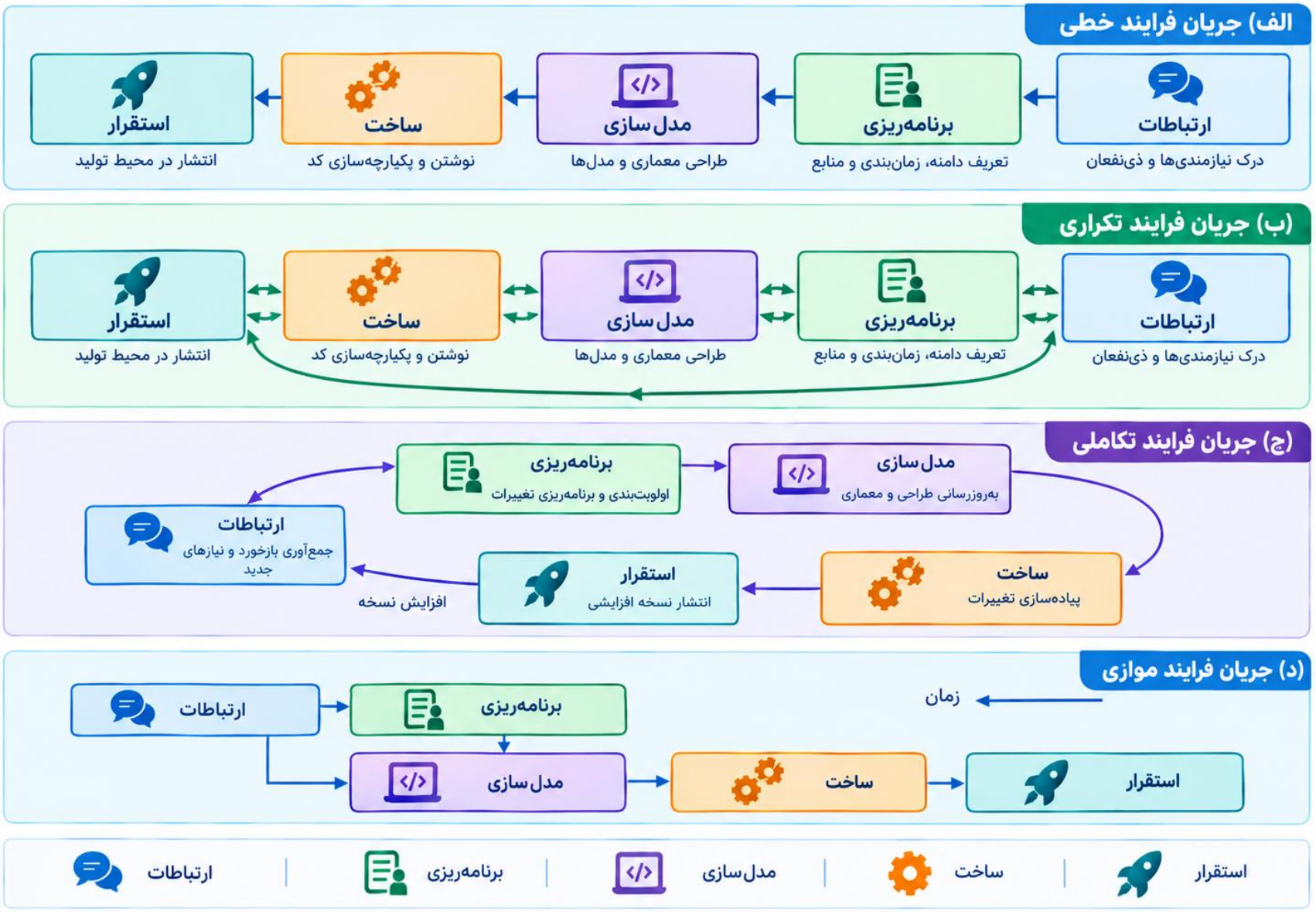
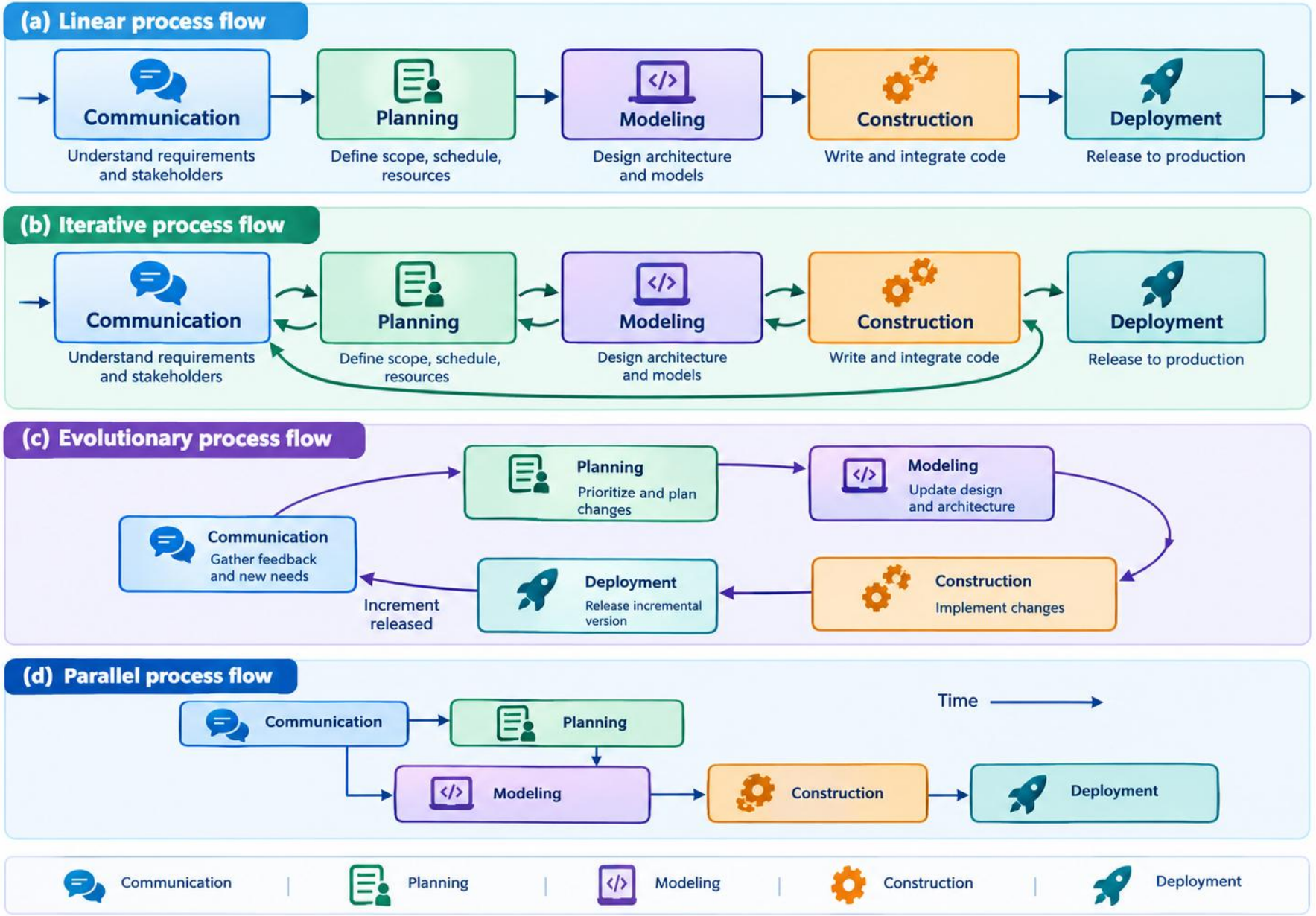


FIGURE 2.2 Software Development Process Flows



مراحل رایج فرآیند توسعه نرم‌افزار (جمع بندی)



یک چارچوب فرآیند عمومی شامل پنج فعالیت است (ادامه)

- 🧠 **برنامه کاربردی پیشنهادی: "دستیار خرید هوشمند AR"**
- 🌟 **ویژگی‌ها:**
- **تجربه خرید واقعیت افزوده :AR Shopping**
  - کاربران می‌توانند لباس، کفش یا وسایل خانه را با دوربین گوشی روی خود یا محیط واقعی امتحان کنند و ببینند چگونه به نظر می‌رسند.
- **توصیه‌های شخصی‌سازی‌شده با هوش مصنوعی:**
  - الگوریتم‌های هوش مصنوعی بر اساس سلیقه، سبک زندگی و تاریخچه خرید، بهترین گزینه‌ها را پیشنهاد می‌دهند.
- **مقایسه قیمت و موجودی لحظه‌ای:**
  - برنامه به‌طور خودکار فروشگاه‌ها و سایت‌های مختلف را بررسی می‌کند و بهترین قیمت و موجودی را نمایش می‌دهد.
- **تعامل اجتماعی و اشتراک‌گذاری:**
  - کاربران می‌توانند خریدها و استایل‌های خود را با دوستان به اشتراک بگذارند و نظرات دوستان را دریافت کنند.
- **پرداخت امن و یکپارچه:**
  - قابلیت پرداخت سریع و امن درون اپلیکیشن با رمزگذاری پیشرفته و احراز هویت بیومتریک.

Dr. A. Taghinezhad

فعالیت‌های چتری در مهندسی نرم‌افزار

این فعالیت‌ها فرآیندهای مهندسی نرم‌افزار را تکمیل و پشتیبانی می‌کنند.

۱ ردیابی و کنترل پروژه نرم‌افزار

ارزیابی پیشرفت در برابر طرح پروژه و حفظ برنامه زمان‌بندی.



۲ مدیریت ریسک

ارزیابی ریسک‌هایی که ممکن است بر نتیجه پروژه یا کیفیت محصول تأثیر بگذارد.



۳ تضمین کیفیت نرم‌افزار

تعریف و انجام فعالیت‌هایی (آزمون‌هایی) برای اطمینان از کیفیت نرم‌افزار.



۴ بررسی‌های فنی

محصولات کاری مهندسی نرم‌افزار را برای کشف و حذف خطاها ارزیابی کنید.



۵ اندازه‌گیری

تعریف و جمع‌آوری نشان‌گرهای کارایی شامل (زمان لود، نرخ شکست، نرخ محاوره با کاربر) که به ارائه نرم‌افزار به نیازهای ذینفعان کمک می‌کند.



- زمان لود
- نرخ شکست
- نرخ محاوره با کاربر

۶ مدیریت پیکربندی نرم‌افزار

اثرات تغییر را در طول فرآیند نرم‌افزار مدیریت کنید.



۷ مدیریت قابلیت استفاده مجدد

معیارهایی را برای استفاده مجدد از محصول کاری تعریف کنید و مکانیسم‌هایی را برای اجزای قابل استفاده مجدد ایجاد کنید.



۸ آماده‌سازی و تولید محصول کاری

انجام فعالیت‌های مورد نیاز برای ایجاد محصولات کاری مانند مدل‌ها، اسناد، گزارش‌ها، فرم‌ها و لیست‌ها.



فعالیت‌های چتری

- فرآیندهای مهندسی نرم‌افزار با مقداری از فعالیت های چتری تکمیل می‌شود که عبارتند از:
- ۱. **ردیابی و کنترل پروژه نرم افزار**: ارزیابی پیشرفت در برابر طرح پروژه و حفظ برنامه زمانبندی.
- ۲. **مدیریت ریسک**: ارزیابی ریسک هایی که ممکن است بر نتیجه پروژه یا کیفیت محصول تأثیر بگذارد.
- ۳. **تضمین کیفیت نرم افزار**: تعریف و انجام فعالیت هایی (آزمون‌هایی) برای اطمینان از کیفیت نرم افزار.
- ۴. **بررسی های فنی**: محصولات کاری مهندسی نرم افزار را برای کشف و حذف خطاها ارزیابی کنید.

فعالیت‌های چتری (ادامه)

- ۵. **اندازه گیری**: تعریف و جمع آوری نشان‌گرهای کارایی شامل (زمان لود، نرخ شکست، نرخ محاوره با کاربر) که به ارائه نرم افزار به نیازهای ذینفعان کمک می کند.
- ۶. **مدیریت پیکربندی نرم افزار**: اثرات تغییر را در طول فرآیند نرم افزار مدیریت کنید.
- ۷. **مدیریت قابلیت استفاده مجدد**: معیارهایی را برای استفاده مجدد از محصول کاری تعریف کنید و مکانیسم هایی را برای اجزای قابل استفاده مجدد ایجاد کنید.
- ۸. **آماده سازی و تولید محصول کاری** : انجام فعالیت های مورد نیاز برای ایجاد محصولات کاری مانند مدل ها، اسناد، گزارش ها، فرم ها و لیست ها.

"انطباق پذیری فرآیند مهندسی نرم افزار"

فرآیند مهندسی نرم‌افزار باید متناسب با نوع پروژه، پیچیدگی مسئله، مهارت تیم و شرایط سازمان تنظیم شود؛ بنابراین، یک فرآیند ثابت برای همه پروژه‌ها مناسب نیست.

نکته کلیدی: فرآیند مهندسی نرم‌افزار یک چارچوب قابل تنظیم است، نه مجموعه‌ای از قوانین تغییرناپذیر؛ هدف، انتخاب مناسب‌ترین فرآیند برای دستیابی به کیفیت مطلوب با هزینه و زمان مناسب است.

Dr. A. Taghinezhad

انطباق‌پذیری فرآیند مهندسی نرم افزار

فرآیند مهندسی نرم‌افزار باید متناسب با نوع پروژه، پیچیدگی مسئله، مهارت تیم و شرایط سازمان تنظیم شود؛
بنابراین، یک فرآیند ثابت برای همه پروژه‌ها مناسب نیست.

پروژه دوم: سامانه بانکی



به دلیل حساسیت امنیتی و مالی، نیاز به مستندسازی دقیق‌تر، آزمون‌های سخت‌گیرانه و کنترل کیفیت بیشتر است.

استقرار و نگهداری → طراحی و معماری → توسعه → آزمون و بازبینی → تحلیل نیازمندی‌ها

- جریان فعالیت‌ها: دقیق‌تر و مرحله‌ای با وابستگی‌های مشخص
- محصولات کاری: مستندات کامل، گزارش‌های کیفیت، کد منبع
- تضمین کیفیت: آزمون‌های امنیتی و کارکردی گسترده
- مشارکت ذی‌نفعان: تعامل با مشتری، کاربران و نهادهای نظارتی
- ساختار تیم: نقش‌های تخصصی (امنیت، تست، تحلیل)، نظارت بیشتر

پروژه اول: فروشگاه اینترنتی ساده



به دلیل ماهیت ساده‌تر و تغییرات سریع، می‌توان از روش چابک (Agile) استفاده کرد.

تحویل تدریجی → تست → توسعه → برنامه‌ریزی اسپرینت

- جریان فعالیت‌ها: ساده و تکرارشونده (اسپرینت‌های کوتاه)
- محصولات کاری: کد، نسخه‌های قابل اجرا، حداقل مستندات
- تضمین کیفیت: آزمون‌های سبک‌تر
- مشارکت ذی‌نفعان: تعامل مداوم با مشتری (بازخورد سریع)
- ساختار تیم: تیم کوچک، نقش‌های منعطف، استقلال بیشتر

مواردی که در فرآیندهای مختلف متفاوت هستند

سازماندهی و نقش‌ها
روش سازماندهی تیم و تعریف نقش‌ها در سازمان

ساختار تیم
نقش‌ها، مسئولیت‌ها و میزان استقلال اعضای تیم

مشارکت ذی‌نفعان
میزان تعامل با مشتری و سایر ذی‌نفعان

تضمین کیفیت
میزان آزمون، بازبینی و کنترل پروژه

محصولات کاری
نوع و میزان مستندات، کد و گزارش‌های موردنیاز

جریان فعالیت‌ها
ترتیب و وابستگی بین فعالیت‌ها، اقدامات و وظایف

نکته کلیدی: فرآیند مهندسی نرم‌افزار یک چارچوب قابل تنظیم است، نه مجموعه‌ای از قوانین تغییرناپذیر؛ هدف، انتخاب مناسب‌ترین فرآیند برای دستیابی به کیفیت مطلوب با هزینه و زمان مناسب است.

لایه ی روش‌ها

- برای توسعه ی هر محصول نرم افزاری یک روش پنج مرحله ای وجود دارد که عبارتند از:
  - تجزیه و تحلیل خواسته ها Requirements Analysis
  - طراحی Design
  - پیاده سازیImplementation
  - آزمون Test
  - پشتیبانی Maintenance

Dr. A. Taghinezhad

تفاوت لایه روش‌ها و فرآیند

- **فرآیند نرم افزار** مجموعه ای از مراحل یا **مجموعه ای از فعالیت هایی است که در طول توسعه نرم افزار** استفاده می شود. برای مثال، اگر می‌خواهیم نرم‌افزار را با موفقیت توسعه دهیم، باید **مجموعه‌ای از قوانین یا فعالیت‌هایی مانند برنامه‌ریزی، جمع‌آوری نیازمندی‌ها، طراحی، پیاده‌سازی، آزمایش و استقرار را دنبال کنیم.**

- **لایه روش ها:** این لایه **دانش فنی** را برای توسعه نرم افزار فراهم می کند که شامل **روش ها، دانش فنی و "چگونگی"** به منظور توسعه نرم افزار است. بیشتر بر روی مرحله خاص در طول توسعه تمرکز دارد.
  - **متدولوژی های مختلفی مانند چابک، آبشار، روش مارپیچی و غیره وجود دارد.**

رویکردهای توسعه نرم‌افزار

هر پروژه می‌تواند با توجه به نیازها، شرایط و نوع محصول، از یک یا ترکیبی از این رویکردها استفاده کند.

۱ آبشاری (Waterfall)

این یک رویکرد خطی و متوالی است که در آن هر مرحله از فرآیند توسعه به ترتیب خاصی انجام می‌شود.



مثال

توسعه یک سامانه بانکی که ابتدا نیازمندی‌ها مشخص می‌شود، سپس طراحی، پیاده‌سازی، آزمون و در نهایت استقرار انجام می‌شود.

۲ چابک (Agile)

این یک رویکرد تکراری و تطبیقی است که پروژه به واحدهای کوچک کار یا «داستان‌های کاربر» تقسیم می‌شود.



مثال

توسعه یک اپلیکیشن فروشگاه اینترنتی با تقسیم به داستان‌های کاربر مانند «ثبت‌نام کاربر»، «مشاهده محصولات» و «پرداخت» و انجام تکرارهای کوتاه برای هر کدام.

۳ اسکرام (Scrum)

این زیرمجموعه‌ای از چابک است که کار را به صورت اسپرینت‌های با زمان محدود سازماندهی می‌کند.



مثال

یک تیم اسکرام در هر اسپرینت، بخشی از محصول را توسعه می‌دهد؛ مثلاً در اسپرینت اول مازول ورود کاربران و در اسپرینت بعدی سیستم پرداخت را پیاده‌سازی می‌کند.

۴ توسعه سریع برنامه (RAD)

این یک رویکرد سریع و انعطاف‌پذیر است که بر نمونه‌سازی سریع و تحویل تکراری تأکید دارد.



مثال

- تیم وارد فازهای نمونه‌سازی سریع می‌شود.
- در هر فاز، بخشی از سیستم را به‌صورت نمونه اولیه می‌سازند و با ذینفعان بررسی می‌کنند تا بازخورد بگیرند.
- پس از تأیید همه بخش‌ها، آن‌ها را در یک سیستم کامل ادغام می‌کنند.

هر رویکرد مزایا و محدودیت‌های خاص خود را دارد و انتخاب بهترین روش به نیاز پروژه، تیم و شرایط سازمان بستگی دارد.

لایه ابزار

- ابزارهای مهندسی نرم افزار پشتیبانی خودکار یا نیمه خودکار را برای فرآیند و روش ها ارائه می دهند.
- این گونه نرم‌افزارها به مهندسین نرم‌افزار کمک می‌کنند تا بتوانند سیستم‌های مورد نیاز را مهندسی نمایند.
- ابزاریهای خودکار اصطلاحا ابزارهای مهندسی نرم افزار به کمک کامپیوتر گفته می شود که ترجمه عبارت "Case Tools" می باشد.
- (Case) computer-aided software engineering
- یکی از معروفترین CASE Tools در دنیا فرآیند یکپارچه ی ارائه شده توسط شرکت Rational که RUP نامیده میشود.

ابزار مهندسی نرم‌افزار (ادامه)

۱ تحلیل و استخراج نیازمندی‌ها

روش‌ها:
مصاحبه با ذی‌نفعان، پرسش‌نامه و جلسات تحلیل نیازمندی‌ها

ابزارها:
Jira · Azure DevOps · Confluence

۲ طراحی و مدل‌سازی نرم‌افزار

زبان مدل‌سازی:
- UML

ابزارها:
StarUML · Visual Paradigm · PlantUML

ترسیم نمودارها:
Microsoft Visio · diagrams.net

۳ برنامه‌نویسی و پیاده‌سازی

محیط‌های توسعه:
- Visual Studio 2026
- Visual Studio Code

مدیریت کد منبع:
Git

میزبانی کد و همکاری تیمی:
GitHub · GitLab

۴ برنامه‌ریزی و مدیریت پروژه

ابزارها:
Jira · Microsoft Planner · Microsoft Project

کاربردها:
- تعریف وظایف • تخمین زمان • تخصیص منابع
- مدیریت اسپرینت‌ها • پیگیری پیشرفت پروژه

۵ آزمون و تضمین کیفیت نرم‌افزار

آزمون خودکار برنامه‌های وب:
- Playwright

آزمون و بررسی APIها:
- Postman

کاربردها:
- شناسایی خطاها • بررسی عملکرد
- ارزیابی کیفیت نرم‌افزار

۶ مستندسازی و مدیریت دانش

ابزارها:
Microsoft Word · Confluence · Markdown

کاربردها:
- تهیه مستندات فنی • مشخصات نیازمندی‌ها
- راهنمای کاربر • گزارش‌های پروژه

۷ یکپارچه‌سازی و استقرار نرم‌افزار

ابزارها:
GitHub Actions · Azure Pipelines

ابزارهای مناسب، توسعه‌ی بهتر، تیم‌های کارآمدتر

لایه ی روش‌ها– تجزیه و تحلیل خواسته ها

- سیستم سنتی که تا کنون حاکم بوده بررسی شود و نیازهای مشتری لیست شود.
- لیست تولید شده در جلسه ای که با مشتری برگزار خواهد شد مورد بازبینی با تیم توسعه قرار گیرد.
- در نیازسنجی باید به اجزا زیر بیشتر دقت کنیم:
  - **ورودی ها و خروجی های** سیستم
  - **پردازشهای که روی آنها** انجام میشود
  - رابط ها : انسان به محیط، انسان به انسان، و محیط با محیط
  - مستندات نیازها

Dr. A. Taghinezhad

لایه روش‌ها: تجزیه‌وتحلیل نیازمندی‌ها

- **هدف این مرحله، شناسایی، تحلیل و مستندسازی نیازهای مشتری و کاربران برای توسعه نرم‌افزاری متناسب با انتظارات آن‌هاست.**

- **بررسی سیستم موجود:** شناسایی مشکلات، محدودیت‌ها و نیازهای سیستم فعلی.
- جمع‌آوری نیازمندی‌ها: تهیه فهرستی از خواسته‌ها و انتظارات مشتریان و کاربران.
- بازبینی و تأیید: بررسی فهرست نیازمندی‌ها با حضور مشتری و تیم توسعه برای رفع ابهام‌ها و تناقض‌ها.
- **تحلیل اجزای اصلی سیستم:**
  - ورودی‌ها و خروجی‌ها: اطلاعات دریافتی و نتایج تولیدشده.
  - پردازش‌ها: عملیات انجام‌شده روی داده‌ها.
  - رابط‌ها و تعاملات: ارتباط انسان با سیستم، افراد با یکدیگر و اجزای مختلف سیستم.
- مستندسازی نیازمندی‌ها: ثبت نیازهای کارکردی و غیرکارکردی به‌عنوان مبنای طراحی، پیاده‌سازی و آزمون نرم‌افزار.
- مثال: در سامانه انتخاب واحد دانشگاه، اطلاعات دانشجو و دروس ورودی، بررسی ظرفیت و پیش‌نیازها پردازش، و برنامه نهایی انتخاب واحد خروجی سیستم است.

Dr. A. Taghinezhad

لایه روش‌ها: طراحی معماری نرم‌افزار

- هدف: تعیین ساختار کلی سیستم، اجزای اصلی، مسئولیت هر جزء، رابط‌ها و نحوه تعامل میان آن‌ها با درنظرگرفتن معیارهایی مانند عملکرد، مقیاس‌پذیری، قابلیت اطمینان و مصرف منابع.
- مراحل اصلی طراحی معماری:

1. تقسیم سیستم به اجزای اصلی.
2. تعیین مسئولیت هر جزء.
3. تعریف رابط‌ها و نحوه ارتباط میان اجزا.
4. بررسی نیازهای عملکردی و غیرعملکردی.
5. انتخاب معماری مناسب و مستندسازی ساختار سیستم.

- مثال: سامانه کتاب‌فروشی آنلاین
- برای پیاده‌سازی سامانه‌ای که امکان جستجو، مشاهده و خرید کتاب را فراهم می‌کند، می‌توان از معماری سه‌لایه استفاده کرد:
- لایه نمایش Presentation رابط کاربری برای جستجو و خرید کتاب.
- لایه منطق کسب‌وکار Business Logic مدیریت سفارش‌ها، محاسبه قیمت و بررسی موجودی کتاب‌ها.
- لایه دسترسی به داده‌ها Data Access ارتباط با پایگاه داده برای ذخیره و بازیابی اطلاعات کتاب‌ها، کاربران و سفارش‌ها.
- نتیجه: تفکیک سیستم به این سه لایه، توسعه، آزمون، نگهداری و گسترش نرم‌افزار را ساده‌تر می‌کند.

لایه ی روش‌ها– آزمون

- در مرحله آزمون سعی میشود
- منطق اجزای داخلی نرم افزار مورد آزمایش قرار گیرد (**آزمودن** تمام **دستورات** برنامه)
- منطق اجزای خارجی عملیاتی برنامه مورد آزمایش قرار گیرد. (**حصول اطمینان از اینکه ورودی های نرم افزار** منجر به ایجاد **خروجی** های درستی میشود.
- انواع آزمون: **تست واحد، تست یکپارچگی، آزمون پذیرش، آزمون کارایی، آزمون امنیتی، آزمون** کاربردپذیری

Dr. A. Taghinezhad

Online Bookstore System: Testing

1.Unit Testing: آزمون واحد

- تک تک اجزا (واحدها) را به صورت مجزا بررسی کنید.
- مثال: الگوریتم جستجوی کتاب را برای نتایج دقیق آزمایش کنید.

2.Integration Testing: آزمون یکپارچگی

- **اعتبار سنجی تعاملات** بین اجزای مختلف
- مثال: اطمینان حاصل کنید که افزودن کتاب به سبد خرید پایگاه داده را به درستی به روز می کند.

3.Acceptance Testing: آزمون پذیرش

- **اعتبارسنجی سیستم در برابر الزامات تجاری**
- **مثال: فرآیند خرید نهایی (جستجو، سبد خرید، پرداخت) را آزمایش کنید.**

Online Bookstore System: Testing

4.Performance Testing: آزمون کارایی

- پاسخگویی، مقیاس پذیری و استفاده از منابع را ارزیابی کنید.
- مثال: زمان پاسخگویی را در اوج ترافیک اندازه گیری کنید..

5.Security Testing: آزمون امنیت

- آسیب پذیری ها را شناسایی کنید و از داده ها محافظت کنید.
- مثال: اطمینان از انتقال ایمن اطلاعات حساس.

6.Usability Testing: آزمون استفاده‌پذیری

- کاربرپسندی را ارزیابی کنید.
- مثال: سهولت پیمایش و جریان خرید را ارزیابی کنید.

Dr. A. Taghinezhad

لایه ی روش‌ها– پشتیبانی

- نگهداری نرم‌افزار *(Software Maintenance)*شامل تغییراتی است که پس از تحویل نرم‌افزار برای رفع خطا، سازگاری، بهبود یا پیشگیری از مشکلات انجام می‌شوند.
- *1.* تصحیحی *(Corrective):*رفع خطاهای موجود در نرم‌افزار .مثال *:*اصلاح محاسبه اشتباه مبلغ سفارش.
- *2.* تطبیقی *(Adaptive)*سازگار کردن نرم‌افزار با تغییرات محیطی .مثال *:*سازگاری با نسخه جدید سیستم‌عامل.
- *3.*بهبوددهنده *(Perfective)*بهبود عملکرد، کیفیت یا افزودن قابلیت‌های جدید .مثال *:* افزودن امکان جست‌وجوی پیشرفته.
- *4.* پیشگیرانه *(Preventive)*کاهش احتمال بروز مشکلات آینده و آسان‌تر کردن نگهداری نرم‌افزار .مثال *:*بازآرایی کد و افزودن آزمون‌های خودکار.
- نکته *:*مهندسی مجدد نرم‌افزار *(Software Reengineering)*می‌تواند به بهبود قابلیت نگهداری کمک کند، اما مترادف نگهداری پیشگیرانه نیست.

مثال: لایه ی روش‌ها– پشتیبانی

- تعمیر و نگهداری نرم افزار برای کتابفروشی آنلاین
  - ۱. تعمیر و نگهداری **اصلاحی**:
  - – خطاها و باگ‌ها را به سرعت برطرف کنید.
  - – مثال: رفع عدم دقت الگوریتم جستجو.
- ۲. نگهداری **پیشگیرانه**:
  - – به طور فعال عمر طولانی را تضمین کنید.
  - – مثال: **درگاه های پرداخت قدیمی را ارتقا دهید.**
- ۳. تعمیر و نگهداری **بهبود دهنده**:
  - – بهبود ویژگی ها و عملکرد.
  - – مثال: بهبود **دقت توصیه کتاب**.– جستجوی فازی در بخش جستجو کتاب‌ها
- ۴. تعمیر و نگهداری **تطبیقی**:
  - – سازگاری با تغییرات (به عنوان مثال، قوانین مالیاتی).
  - – مثال: به روز رسانی ماژول قیمت گذاری.

Dr. A. Taghinezhad

اصول کلی مهندسی نرم‌افزار

نرم‌افزار خوب، فقط کد نیست؛ نتیجه‌ی تصمیم‌های درست است.

1 ایجاد ارزش

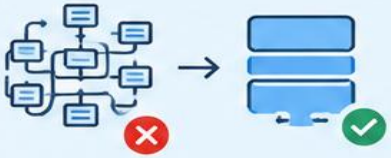
هر قابلیت باید نیاز واقعی کاربر را برآورده کند.



مثال:
افزودن تغییر رنگ برای بهبود دسترس‌پذیری.

2 سادگی (KISS)

از پیچیدگی غیرضروری پرهیز کنید.



مثال:
انتخاب معماری ساده و متناسب با نیاز پروژه.

3 حفظ چشم‌انداز

تصمیم‌های فنی باید با اهداف محصول هماهنگ باشند.



4 توجه به دیگران

کد و مستندات را خوانا، قابل‌فهم و قابل‌نگهداشت بنویسید.



5 آمادگی برای تغییر

طراحی را برای تغییرات محتمل انعطاف‌پذیر نگه دارید.



6 استفاده‌ی مجدد

در صورت مناسب بودن، از مؤلفه‌ها و راه‌حل‌های معتبر موجود استفاده کنید.



7 تفکر پیش از اقدام

پیش از پیاده‌سازی، مسئله و پیامدهای راه حل را بررسی کنید.



بهتر فکر کنیم / بهتر کنیم / ماندگارتر باشیم

</>

- پایان بخش اول

Dr. A. Taghinezhad